Programación Orientada a Objetos (POO) en C++ — Juego de Naves en Consola (class Nave)

Alexis Maier
E.E.S.T. N° 1 “Eduardo Ader” — Vicente López
Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B
Profesor: Mansilla Muñoz York Elías
Fecha: 21 de octubre de 2026

Introducción

La actividad 8 introduce la Programación Orientada a Objetos como la evolución natural del *struct*: una *class* agrupa atributos (datos) y métodos (acciones) dentro de una misma "caja fuerte" llamada objeto, aplicando encapsulamiento mediante *private* y *public*. Se desarrolla un simulador de combate espacial en consola con las clases *Nave*, *Asteroide* y *Bala*, controlado por teclado (W, A, S, D y ESPACIO).

Desarrollo

La clase *Nave* encapsula la posición X/Y y las vidas, exponiendo métodos públicos para moverse, pintar, borrar y recibir daño. El motor principal gestiona un vector de asteroides que descienden y otro de balas que ascienden, evaluando colisiones asteroide-nave y bala-asteroide, re-spawneando los objetos. El HUD superior muestra el nombre del piloto y las vidas restantes. Como funcionalidad extra del motor se incluyen el disparo, las colisiones y el HUD con el nombre del jugador.

Cuestionario Técnico de Control

1. **struct vs class:** La principal diferencia técnica es que en un *struct* los miembros son públicos por defecto, mientras que en una *class* son privados por defecto. La clase agrega encapsulamiento (*private*/*public*) y métodos asociados.

2. **Encapsulamiento:** Dejar la salud como pública sería peligroso porque cualquier código externo podría asignar valores inválidos (ej. salud negativa), rompiendo las reglas del juego; por eso se protege y se modifica solo mediante *reducirVida()*.

3. **Constructor:** El bloque *Nave(int, int, int)* inicializa los atributos privados cuando se crea el objeto, garantizando un estado válido desde el instante de su construcción.

4. **Desafío Funcionalidad:** Se implementó el motor completo de combate: movimiento con W/A/S/D, disparo con ESPACIO, detección de colisiones, re-spawn de asteroides y HUD con nombre del piloto. La lógica se insertó en el bucle principal tras la creación de los objetos.

Conclusión

La POO permite modelar entidades del mundo real (naves, asteroides, balas) con datos y comportamiento encapsulados, mejorando la mantenibilidad y reutilización del código para el Proyecto Integrador Anual.

Referencias

cplusplus.com. (s.f.). Classes and Objects in C++. https://cplusplus.com/doc/tutorial/classes/

GeeksforGeeks. (s.f.). Access modifiers in C++. https://www.geeksforgeeks.org/access-modifiers-in-c/

Microsoft Learn. (s.f.). Constructores e inicialización de clases. https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp